Versión 1.2

Paradigmas de Programación

Paradigma Funcional

ING. DAVID CECCHI
ING. JACOBO DOMINGUEZ
ING. NICOLÁS TIZZANO

UNIVERSIDAD TECNOLOGICA NACIONAL
Facultad Regional San Nicolás
Ingeniería en Sistemas

Paradigma Funcional

El cálculo lambda, desarrollado por Alonzo Church en la década de 1930, es un sistema formal que sienta las bases teóricas de la programación funcional.
A pesar de su simplicidad sintáctica, el cálculo lambda puede representar cualquier función computable.
Se caracteriza por tratar la computación como la evaluación de funciones matemáticas puras, evitando los cambios de estado mediante la inmutabilidad, utilizando funciones de orden superior, recursividad y composición de funciones.
La programación funcional pura tiene las siguientes características:

- No hay variables.
- Sólo constantes, parámetros y valores.
- No hay estatutos de asignación.
- No hay iteración, solo recursión.
- No hay noción del estado interno de una función: el valor de cualquier función depende sólo de los valores de sus parámetros y no de cualquier cómputo previo, incluyendo llamadas a la propia función.

En el paradigma funcional, un **programa** se expresa a través de un conjunto de definiciones de funciones puras: funciones que ante los mismos datos de entrada producen los mismos resultados sin recurrir a cambios de estados. La ejecución de un programa funcional consiste en un proceso de reducción progresiva de expresiones mediante la sustitución de argumentos en el cuerpo de las funciones definidas, hasta alcanzar un valor irreducible.
Se selecciona al lenguaje Haskell para la implementación del paradigma. Con el objetivo de facilitar la interpretación de conceptos se recurrirá a ejemplos prácticos, necesitando para ello iniciar con el detalle de los tipos de datos, las funciones predefinidas y la sintaxis del lenguaje.

Tipos de Datos

Tipos Primitivos

Tipo de Dato	Descripción
Int	Entero de precisión fija (64 bits), con signo
Integer	Entero de precisión arbitraria (sin límite: limitado por la memoria), con signo.
Word	Entero de precisión fija (64 bits), sin signo.
Float	Precisión simple
Double	Precisión doble
Bool	Booleano
Char	Carácter
String	Alias de [Char]

Tipos Especiales

Tipo de Dato	Descripción
a	Es una **variable de tipo**, admite todos los tipos de datos
Num	Es una **clase de tipo**, admite tipos numéricos: Int, Integer, Double, Float, etc.
Integral	Es una **clase de tipo**, admite tipos enteros: Int, Integer y Word.
Fractional	Es una **clase de tipo**, admite tipos fraccionarios: Double, Float y Rational.

Listas

Una lista es una serie de elementos del mismo tipo. Los elementos se encierran entre corchetes y se encuentran separados por comas. Se admite las listas de listas. Ejemplos:

- [21, 17, 34]
- [“Martillo”, ”Pinza”, ”Destornillador”]
- [[18,52], [1,2,3,4,5,6,7], [89,56,34]]

Los String son listas de chars: “Juan” representa a [‘J’, ‘u’, ‘a’, ‘n’]

Para generar una lista de enteros consecutivos, no es necesario enumerar todos los elementos, basta con especificar un rango separando el límite inferior y superior a través de dos puntos:

- [1..10] representa la lista [1,2,3,4,5,6,7,8,9,10]
- [20, 3..7] representa la lista [20,3,4,5,6,7]
- [20, 7.. -5] representa la lista [20,7,6,5,4,3,2,1,0,-1,-2,-3,-4,-5]
- [1..] lista infinita, representa la lista [1,2,3,4,5,…….]

Patrón	Descripción
[]	Lista vacía
(x:xs)	El operador : separa la cabeza de la cola de la lista. Ejemplo: Para [21, 14, 18, 34] la cabeza es 21 y la cola [14, 18, 34] Para [76] la cabeza es 76 y la cola es una lista vacía []. Para [] no se identifica cabeza ni cola.
(x:y:ys)	Lista de al menos dos elementos. Ejemplo: Para [21, 14, 18, 34] se identifica x=21, y=14, ys=[18,34] Para [76,11] se identifica x=76, y=11, ys=[] Para [1] no se identifica patrón.
[x]	Lista de un solo elemento
[x, y]	Lista de exactamente dos elementos

Funciones	Descripción
head xs	Retorna el primer elemento (cabeza) de la lista
tail xs	Retorna una lista sin el primer elemento (cola)
last xs	Retorna el último elemento de la lista
xs !! n	Retorna el elemento de la posición n. La primer posición es cero.
elem x xs	Retorna True si el elemento x se encuentra en la lista.
x : xs	Agrega el elemento x al inicio de la lista
take n xs	Retorna una lista con los primeros n elementos
drop n xs	Retorna una lista sin los primeros n elementos
xs ++ ys	Retorna la concatenación de ambas listas
length xs	Retorna la longitud de la lista
minimun xs	Retorna el menor elemento de la lista
maximun xs	Retorna el mayor elemento de la lista
sum xs	Retorna la suma de los elementos de una lista numérica
map f xs	Retorna una copia de la lista xs habiendo aplicado la función f a cada uno de sus elementos

filter f xs	Retorna una copia de la lista xs con los elementos para los cuales la función f toma valor True, es decir, aquellos elementos que verifican la condición.
all f xs	Retorna True si todos los elementos de la lista cumplen con la función f, es decir, verifican la condición.
any f xs	Retorna True si alguno de los elementos de la lista cumple con la función f, es decir, verifica la condición.
foldr f z xs	Retorna un único valor que resulta de combinar recursivamente los elementos de xs con una función binaria f: se aplica f entre el primer elemento y el resultado de aplicar foldr (con la misma f y z) al resto de la lista. Cuando la lista es vacía, el resultado es directamente z. main :: IO () main = do print (foldr (&&) True [True, True, False]) --False print (foldr (&&) True []) --True print (foldr (+) 1 [1,2,3]) --7

Ejemplo

```
duplicarLista :: [Int] -> [Int]
duplicarLista datos = map (*2) datos

main :: IO()
main = do
  let miLista = [1,7,15]
  print (duplicarLista miLista) -- se presenta [2, 14, 30]
```

Tuplas

Las tuplas representan un tipo de dato compuesto, es decir, sus elementos pueden ser de diferentes tipos. Los elementos de una tupla se encierran entre paréntesis.

Tupla	Tipo
(17, 3, 1990)	(Int, Int, Int)
(“Lucas”, 17)	(String, Int)
(23, [17, 28, 32], “una casa”)	(Int, [Int], String)
(‘a’, False)	(Char, Bool)
((23, 17), (98, 56, 34))	((Int, Int), (Int, Int, Int))

Funciones	Descripción
fst xs	Retorna el primer elemento de la tupla
snd xs	Retorna el segundo elemento de la tupla

Ejemplo

```
sumaComplejo :: (Int, Int) -> (Int, Int) -> (Int, Int)
sumaComplejo (ra, ia) (rb, ib) = (ra + rb, ia + ib)

main :: IO()
main = do
   let resul = sumaComplejo (2, 3) (4, 5)
```

```
   print (resul) -- se presenta el nro. complejo (6, 8)
```

Ejemplo versión 2

```
sumaComplejo :: (Int, Int) -> (Int, Int) -> (Int, Int)
sumaComplejo (ra, ia) (rb, ib) = (ra + rb, ia + ib)

main :: IO()
main = do
   let (real, ima) = sumaComplejo (2, 3) (4, 5)
   print (show real ++ ", " ++ show ima ++ "i") -- se presenta el nro. complejo “6, 8i”
```

Ejemplo versión 3

```
sumaComplejo :: (Int, Int) -> (Int, Int) -> (Int, Int)
sumaComplejo (ra, ia) (rb, ib) = (ra + rb, ia + ib)

parteReal :: (Int, Int) -> Int
parteReal (n,_) = n

parteIma :: (Int, Int) -> Int
parteIma (_,n) = n

main :: IO()
main = do
   let resul = sumaComplejo (2, 3) (4, 5)
   print (show (parteReal resul) ++ ", " ++ show (parteIma resul) ++ "i")
-- se presenta el nro. complejo “6, 8i”
```

Tipos Personalizados

Es posible definir enumeraciones y nuevos tipos de datos que se agreguen a los ya existentes en el lenguaje, mediante la instrucción ***data***. Para ganar expresividad, es posible recurrir a la sintaxis *record syntax* que obliga a describir sobre cada campo su nombre y tipo asociado, encerrándolos entre llaves.

Al definir un tipo con record syntax, el compilador genera automáticamente una función de acceso (accessor) para cada campo, permitiendo obtener su valor directamente. Además, al instanciar se podrá utilizar record syntax nombrando los campos en cualquier orden, o bien por posición, como en un tipo definido sin record syntax; en ese caso, los valores deberán respetar el orden de la definición.

Sintaxis

1. Enumeraciones o Constructores Sin Campos

```
data NombreDelTipo = NombreEnum1 | NombreEnum2 | NombreEnumN deriving (Show)
```

2. Constructores sin Record Syntax

```
data NombreDelTipo
= ConstructorA TipoCampoA1 TipoCampoA2 TipoCampoAN
| ConstructorB TipoCampoB1 TipoCampoB2 TipoCampoBN
| ConstructorN TipoCampoN1 TipoCampoN2 TipoCampoNN deriving (Show)
```

3. Constructores con Record Syntax

```
data NombreDelTipo
= ConstructorA
{NombreCampoA1 :: TipoCampoA1, NombreCampoA2 :: TipoCampoA2, NombreCampoAN :: TipoCampoAN }
| ConstructorB
{NombreCampoB1 :: TipoCampoB1, NombreCampoB2 :: TipoCampoB2, NombreCampoBN :: TipoCampoBN }
```

```
| ConstructorN
{NombreCampoN1 :: TipoCampoN1, NombreCampoN2 :: TipoCampoN2, NombreCampoNN :: TipoCampoNN }
deriving (Show)
```

1. Ejemplo Enumeración con Comparación

```
--para poder comparar (tono == Rojo) debe derivar de Eq
data Color = Rojo | Verde | Azul deriving Eq

puntaje :: Color -> Int -> Int
puntaje tono cantidad
 | tono == Rojo = cantidad * 2
 | tono == Verde = cantidad * 5
 | tono == Azul = cantidad * 10

main :: IO()
main = do
  print (puntaje Rojo 3)
  print (puntaje Verde 6)
  print (puntaje Azul 8)
```

2. Ejemplo Básico sin Record Syntax

```
data Catedra = Catedra String Int [String] (Int, Int, Int) Bool deriving (Show)

mesInicio :: (Int, Int, Int) -> Int
mesInicio (_, mes, _) = mes

-- al no implementar Record Syntax se requiere definir funciones para acceder a los campos de manera individual
nivel :: Catedra -> Int
nivel (Catedra _ niv _ _ _) = niv

primeraClase :: Catedra -> (Int, Int, Int)
primeraClase (Catedra _ _ _ pri _) = pri

main :: IO()
main = do
  let matParadigmas = Catedra
        "Paradigmas"
        2
        ["Cecchi","Tizzano","Dominguez"]
        (27,3,2026)
        True
  print (matParadigmas)
  print (nivel matParadigmas)
  print (mesInicio (primeraClase matParadigmas))
```

3. Ejemplo Básico con Record Syntax

```
data Catedra = Catedra {
nombre :: String,
nivel :: Int,
docentes :: [String],
primeraClase :: (Int, Int, Int),
esTroncal :: Bool
```

```
} deriving (Show) --  si se desea que el tipo de dato cuente con la posibilidad de convertirse a String
                      de forma automática (ej. para mostrar por consola), debe derivar de Show

mesInicio :: (Int, Int, Int) -> Int
mesInicio (_, mes, _) = mes

main :: IO()
main = do
 --instanciando con Record Syntax
 let matParadigmas = Catedra {
       nombre = "Paradigmas",
       nivel = 2,
       docentes = ["Cecchi","Tizzano","Dominguez"],
       primeraClase = (27,3,2026),
       esTroncal = True
       }
 --instanciando sin Record Syntax
 let matAlgoritmia = Catedra
       "Algoritmos"
       1
       ["Rolandelli","Torrez","Mendoza"]
       (26,3,2026)
       False
 print (matAlgoritmia)
 print (matParadigmas)
 print (nivel matParadigmas)
 print (mesInicio (primeraClase matParadigmas))
```

4. Ejemplo Mejorado

```
data Fecha = Fecha {
 dia :: Int,
 mes :: Int,
 anio :: Int
} deriving (Show)

data Catedra = Catedra {
nombre :: String,
nivel :: Int,
docentes :: [String],
primeraClase :: Fecha,
esTroncal :: Bool
} deriving (Show)

main :: IO()
main = do
 let matParadigmas = Catedra {
 nombre = "Paradigmas",
 nivel = 2,
 docentes = ["Cecchi","Tizzano","Dominguez"],
 primeraClase = Fecha 27 3 2026,
 esTroncal = True
 }
 print (matParadigmas)
```

```
  print (nivel matParadigmas)
  print (mes (primeraClase matParadigmas))
```

Realizar modificaciones sobre los valores de las propiedades de un objeto no está permitido: estaría contradiciendo el concepto de inmutabilidad. La estrategia consiste en generar una nueva versión del objeto original que contenga los nuevos valores y manteniendo aquellos que no sufrieron cambios.

Ejemplo

```
data Estudiante = Estudiante {
   nombre :: String,
   legajo :: Int,
   nota   :: Int
} deriving (Show)

main :: IO()
main = do
  let aluOriginal = Estudiante "Juan" 200 10
  let aluNuevo = aluOriginal {nota = 7}
  print (aluNuevo)
  print (aluOriginal {legajo = 300} {nombre = "Luis"})
```

Funciones Predefinidas Básicas

Funciones	Descripción
x + y	Suma
x - y	Resta
x * y	Producto
x / y	División
x ^ y	Potencia
mod x y	Resto de la división
x > y	Mayor que
x >= y	Mayor o igual que
x < y	Menor que
x <= y	Menor o igual que
x == y	Igual que
x /= y	Distinto que
&&	And
||	Or
not x	Negación
x ++ y	Concatena listas y cadenas.
even x	Retorna true si el valor es par
round x	Retorna el redondeo al entero más cercano
abs x	Retorna el valor absoluto
show x	Retorna en formato de texto el valor de x
print x	Imprime por consola el valor de x, **sin importar su tipo**.
putStrLn	Imprime por consola **solo cadenas de texto**, requiere conversión de los datos a tipo String.

Los operadores pueden clasificarse como infijos o prefijos. Haskell plantea una convención sintáctica: los símbolos son infijos mientras que las palabras son prefijos.

Ejemplos de Operadores Prefijos

1. div 10 2
2. mod 7 3
3. max 4 9
4. map f xs

Ejemplos de Operadores Infijos

1. 2 + 3
2. 10 * 5
3. [1,2] ++ [3,4]
4. True && False
5. 10 >= 6

El operador menos "–", representa un caso especial, f -3 es interpretado como (f) – (3) y no como f (-3)

Es posible implementar un operador infijo como prefijo y viceversa, para ello los infijos deben encerrarse entre paréntesis y los prefijos encerrarse entre comillas simples.

Ejemplos de Operadores Prefijos como Infijos

1. 10 `div` 2
2. 7 `mod` 3
3. 4 `max` 9
4. f `map` xs

Ejemplos de Operadores Infijos como Prefijos

1. (+) 2 3
2. (*) 10 5
3. (++) [1,2] [3,4]
4. (&&) True False
5. (>=) 10 6

Es posible definir un operador infijo con una aplicación parcial, consiste en encerrarlo ente paréntesis con solo uno de los parámetros, derecha o izquierda.

```
1. 3 * 4    -- forma infija
2. (*) 3 4  -- forma prefija
3. (* 2) 3  -- sección derecha, equivale a 3 * 2
4. (2 *) 3  -- sección izquierda, equivale a 2 * 3
```

Funciones de Conversión Básicas

Funciones	Descripción
show valor	Convierte a texto cualquier valor cuyo tipo sea instancia de la clase Show, lo que incluye todos los tipos primitivos. Puede convertir listas y tuplas si sus elementos son instancias de la clase Show.

	Puede convertir los tipos propios que derivan de Show (deriving Show).
read texto :: tipoDestino	Convierte texto al tipo requerido. Lanza excepción en caso de error.
fromIntegral valor :: tipoDestino	Convierte un número entero al tipo requerido. Lanza excepción en caso de error.
round valor	Convierte un número a entero. Lanza excepción en caso de error.

Cuando no se especifica el tipo de retorno, se lo infiere a partir del contexto de uso (el tipo de retorno de la función que contiene la expresión, la operación en que se emplea el resultado, o el tipo esperado por otra función). Si el contexto no alcanza, fromIntegral recurre a un tipo por defecto (Integer) y read produce un error de ambigüedad.

Ejemplo

```
main :: IO()
main = do
  print ("de texto a int: " ++ show (read "3" :: Int)) -- "de texto a int: 3"
  print ("de texto a double: " ++ show (read "3.57" :: Double)) -- "de texto a double: 3.57"
  print ("de texto a bool: " ++ show (read "False" :: Bool)) -- "de texto a bool: False"
  print ("de int a double: " ++ show (fromIntegral 3 :: Double)) -- "de int a double: 3.0"
  print ("de float a int: " ++ show (round 3.57)) -- "de float a int: 4"
```

Algunas Reglas Sintácticas

- Mantener la identación, todo aquello que pertenezca al mismo bloque debe comenzar en la misma columna, para ello se deben utilizar espacios (no tabuladores).
- Si una línea continúa en la siguiente, debe estar más identada que la columna de referencia.
- Mantener un espacio antes y después de cada operador.
- Las cadenas se encierran entre comillas dobles, los caracteres entre comillas simples.
- Para comentar una línea se utilizan dos guiones medios al inicio de ella.
- Para comentar múltiples líneas, el bloque debe encerrarse entre llaves y guiones medios {- … -}.
- Los identificadores comienzan con una minúscula o un guion bajo.
- Los tipos (Int, String, Char, etc.) y constructores de datos (True, False) comienzan con mayúscula.
- Los paréntesis solo se utilizan para definir el orden de evaluación o agrupar expresiones.
- Las funciones no utilizan paréntesis para encerrar sus argumentos, salvo situaciones especiales donde los argumentos conforman una expresión compuesta: f x + 1 aplica f a x y luego suma 1, mientras que la expresión f (x + 1) aplica f a (x + 1).
- La aplicación de funciones se define a través de un espacio entre ellas y se evalúa de izquierda a derecha: f x y se evalúa como (f x) y.

Vocabulario

Forma Normal (NF) o Irreductibilidad
La expresión está completamente evaluada, sin operaciones pendientes.
Cualquier intento de "seguir evaluando" devuelve la misma expresión: no puede seguir siendo reducida/simplificada**.**

Ejemplos
1. 42 ← NF
2. (1, "hola") ← NF
3. [1, 2, 3] ← NF

Aplicación
Es el acto *sintáctico* de escribir una función junto a sus argumentos. El operador que identifica la aplicación de función es el espacio en blanco y es el de mayor precedencia. La expresión “f x” describe "aplicar f a x". Pero nada se computa hasta que comience su evaluación.

Evaluación
Es el proceso de reducir la expresión hasta obtener un resultado: computar.

```
doble :: Int -> Int
doble x = x * 2
```
```
doble 5     ← APLICACIÓN (solo una expresión escrita)
  ↓
 5 * 2      ← EVALUACIÓN (ocurre al reducirla)
  ↓
  10        ← FORMA NORMAL (ya no puede reducirse más)
```

```
let x = doble 100    ← APLICACIÓN: se construye la expresión, NO se evalúa todavía.
```
```
print x              ← EN ESTE MOMENTO Haskell se ve forzado a EVALUAR (recién ahora reduce)
```

Definición de Funciones

La sintaxis para definir una función es la siguiente:

*nombrefunción :: tipoParam1 -> tipoParamN -> **tipoRetorno***
nombrefunción nombreParam1 nombreParamN = bloqueInstrucciones

Ejemplificando:
```
alumnoAprobado :: Int -> Bool
alumnoAprobado nota = nota >= 6
```

Aplicando:
```
alumnoAprobado 7
```

Evaluando:
```
7 >= 6
```

Valor de retorno:
```
True
```

Al enviar parámetros a una función, el compilador evaluará la expresión hasta obtener un valor que es irreductible para Haskell.
No es obligatorio definir el tipo de la función dado que el motor es capaz de inferirlos a partir de la implementación.
El compilador realizará los controles pertinentes para corroborar que los datos brindados, y cómo los tipos definidos pueden ser procesados. Ejemplificando:

1. Si se define la función:

```
producto x = “casa” * x
```

   Detectará que “casa” no puede aplicarse con el operador de multiplicación.

2. Si se define la función:

```
producto :: String -> Int
producto x = 3 * x
```

   Detectará que un String no puede aplicarse con el operador de multiplicación.

3. Si se define la función:

```
producto :: Int -> Int
producto x = 3 * x
```

   Y se intenta evaluar:

```
producto “6”
```

   Detectará que un “6” no coincide con el tipo del parámetro requerido (Int).

Es posible definir funciones por partes, para ello se recurren a las guardas (símbolo |):

```
mayor x y  | x > y = x
           | otherwise = y
```

Como otherwise siempre se cumple, debe definirse en última instancia.
La línea de otherwise debe comenzar con un espacio para que haskell interprete correctamente la definición de la función.
En este caso, como el operador > puede aplicar sobre diversos tipos (int, string, bool), la función puede utilizarse con cualquier tipo de dato (siempre que ambos parámetros sean del mismo tipo).

Ejemplos

```
1.  mayor 10, 5              -- valor de retorno: 10
2.  mayor “pepe” “juliana”   -- valor de retorno: “juliana”
3.  mayor true false         -- valor de retorno: true
4.  mayor 4 “juliana”        -- no válido, no puede comprar int con string
```

Al definir funciones por partes, se debe tener presente si realmente tiene sentido estructurarla de esa manera.

Estructura inapropiada

```
puedoAvanzar :: String -> Bool
puedoAvanzar color | color == "verde" = True
                   | otherwise = False
```

Estructura apropiada (correcta)

```
puedoAvanzar :: String -> Bool
puedoAvanzar color = color == "verde"
```

Instrucción Let

La instrucción Let permite vincular de manera inmutable una expresión o valor a un identificador. No representa una asignación en términos del paradigma imperativo.

Sintaxis

```
let <definiciones>
in <expresión que usa esas definiciones>
```

Ejemplo donde las definiciones establecidas por let existen en todo el bloque.

```
let x = 100    -- instrucción dentro de do
    y = 200    -- instrucción dentro de do
print (x + y)  -- instrucción dentro de do
```

Ejemplo anterior traducido por el compilador.

```
let x = 100          -- instrucción dentro de do
    y = 200          -- instrucción dentro de do
  in print (x + y)   -- instrucción dentro de do
```

Ejemplo donde las definiciones establecidas por let existen solo para "in".

```
resultado :: Int
resultado = let  x = 100    -- definición local
                 y = 200    -- definición local
            in x + y        -- expresión que utiliza las definiciones dadas
```

Ejemplo para visualizar el enlace/vinculación en el paradigma funcional (no asignación)

```
decena :: Int -> Int
decena valor = valor + 10

centena :: Int -> Int
centena valor = valor + 100

main :: IO()
main = do
  let a = decena 5   --"a" se enlaza/vincula a la expresión: decena 5
  let b = a          --"b" se enlaza/vincula a la expresión referenciada por "a": decena 5
  let a = centena 7  --"a" se enlaza/vincula a la expresión: centena 7
  print (a)          --retorna 107
  print (b)          --retorna 15
```

Cláusula Where

La cláusula where permite definir identificadores locales, vinculados de forma inmutable a expresiones o funciones auxiliares, cuyo ámbito de validez se extiende a toda la definición de la función en la que se declara, incluyendo ecuaciones o guardas.

Sintaxis

```
nombreFuncion parametros = expresionPrincipal
    where
        definicion1 = expresion1
        definicion2 = expresion2
```

1. Ejemplo Básico:

```
areaCirculo :: Double -> Double
areaCirculo radio = pi * radio ^ 2
    where
        pi = 3.14159      -- pi solo existe dentro de areaCirculo
```

2. Ejemplo Básico Ampliado:

```
medioVolumenCilindro :: Double -> Double -> Double
medioVolumenCilindro radio altura = volumenCilindro * 0.5
    where
        pi = 3.14159
        areaCirculo = pi * radio ^ 2
        volumenCilindro = areaCirculo * altura
```

3. Ejemplo con Definición de Función:

```
volumenCilindro :: Double -> Double -> Double
volumenCilindro radio altura = areaCirculo radio * altura
    where
        areaCirculo :: Double -> Double --  se define función areaCirculo
        areaCirculo r = pi * r ^ 2              solo accesible desde volumenCilindro
            where
                pi = 3.14159 – solo accesible desde areaCirculo
```

4. Ejemplo con Guardas

```
calcularPeso :: Double -> String -> Double
calcularPeso masa astro = masa * factor
    where
        factor
            | astro == "luna" = 1.62
            | astro == "marte" = 3.71
            | astro == "jupiter" = 24.79
            | otherwise = 9.8
```

5. Ejemplo con Definición de Función con Guardas

```
clasificarIMC :: Double -> Double -> String
clasificarIMC pesoKg alturaMts = tipo indice
    where
        indice = pesoKg / alturaMts ^ 2
        tipo :: Double -> String
        tipo valor
```

```
                    | valor < 18.5 = "bajo peso"
                    | valor < 25.0 = "normal"
                    | valor < 30.0 = "sobrepeso"
                    | otherwise = "obesidad"
```

6. Ejemplo con Guardas en Función Principal

```
clasificarIMC :: Double -> Double -> String
clasificarIMC pesoKg alturaMts
    | indice < 18.5 = "bajo peso"
    | indice < 25.0 = "normal"
    | indice < 30.0 = "sobrepeso"
    | otherwise = "obesidad"
   where
        indice = pesoKg / alturaMts ^ 2    -- índice compartido por todos los guards
```

Si las guardas definen a la función principal (devuelven directamente el resultado) es conveniente definirlas en ella, si las guardas (o condicional) solo definen una de las piezas de la función entonces es conveniente definirlas en la cláusula where.

Ejemplificando, misma resolución empleando let y where

```
main :: IO()
main = do
 let resultado = let x = 10
                     y = 20
                 in x + y
 print resultado

main :: IO()
main = do
 let resultado = x + y
                 where
                    x = 10
                    y = 20
 print resultado
```

Conceptos

Funciones puras

Una función pura siempre devuelve el mismo resultado para los mismos argumentos y no produce efectos secundarios al evaluarse: no modifica variables externas, no realiza operaciones de entrada/salida ni altera el estado del sistema.

En Haskell, toda función es pura, aunque existen mecanismos excepcionales (ej.: unsafePerformIO) que rompen esa afirmación y por lo tanto se desaconseja su uso.

Una firma corresponde a una función cuando recibe al menos un parámetro, es decir, cuando su tipo tiene una flecha (->) que no está encerrada dentro de otro tipo (IO, lista, tupla o Maybe). Si no recibe ningún parámetro, la firma corresponde a un valor. Por ejemplo, *f :: IO (Int -> Int)* es un valor de tipo acción, mientras que *f :: (Int -> Int)* es una función (estos paréntesis solo agrupan: no encierran la flecha dentro de otro tipo, por lo que se interpreta como *f :: Int -> Int*).

Según el tipo de resultado que devuelve, se distinguen dos categorías de funciones puras:

- **Funciones puras que retornan un valor común**: aquellas que retornan, por ejemplo, un Int o un String.

- **Funciones puras que retornan una acción**: aquellas que retornan una acción, es decir, un valor de tipo IO. Estas no producen efecto al evaluarse: el efecto ocurre cuando la acción devuelta se ejecuta. *Devolver una acción no implica su ejecución. Una acción se ejecuta únicamente cuando forma parte de las acciones que corren en el main*.

Por otro lado, una firma que no cumple con los criterios asociados a una función, según su tipo se clasifica en dos categorías:

- **Valor común**: representa una constante, siempre expresa el mismo valor.
- **Valor de tipo acción**: describe un cómputo que puede interactuar con el entorno y producir un resultado; sus efectos ocurren cuando la acción se ejecuta (por ejemplo, *saludar :: IO ()*, definida como *saludar = putStrLn "Hola"*, escribe un mensaje en pantalla al ejecutarse).

Firma	Clasificación
f1 :: Double -> Double	Función pura que retorna un Valor Común
f2 :: String -> IO ()	Función pura que retorna una Acción
f3 :: String	Valor Común (constante)
f4 :: IO Double	Valor de Tipo Acción (Acción)

1. Función Pura que Retorna un Valor Común:

```
suma :: Int -> Int -> Int
suma x y = x + y
suma 3 5   -- Siempre retornará 8 (no modifica variables globales, no imprime, no lee archivos, etc.)
```

2. Acción:

```
saludarUsuario :: IO ()
saludarUsuario = do
      putStrLn "¿Cuál es tu nombre?"
      nombre <- getLine
      putStrLn ("Hola, " ++ nombre ++ "!")

main :: IO()
main = saludarUsuario
```

- Se define una acción de Tipo Valor IO (acciones con efecto en el mundo exterior)
- DO define el bloque que contiene el conjunto de acciones a ejecutar, si solo se requiere una acción entonces DO no es necesario
- GETLINE es una acción de Tipo de Valor IO (no una función), al ser requerida, el runtime la ejecutará para obtener el texto ingresado por el usuario

3. Función Pura que Retorna una Acción:

```
import System.Random (randomRIO)

aleatorioEntre :: Int -> Int -> IO Int
aleatorioEntre minimo maximo = randomRIO (minimo, maximo)

main :: IO ()
main = do
 n <- aleatorioEntre 10 20
 putStrLn ("Número obtenido: " ++ show n)
```

- Generación de un número aleatorio dentro de un intervalo

4. Función Pura que Retorna un Valor Común:

```
import Data.Time.Clock
mensajeTiempo :: String -> UTCTime -> String
mensajeTiempo nombre fechaHora = nombre ++ " " ++ show fechaHora

main :: IO ()
main = do
    putStrLn "ingrese su nombre: "
    usuario <- getLine
    diahora <- getCurrentTime
    putStr (mensajeTiempo usuario diahora)
```

- mensajeTiempo es una función pura porque la hora la obtiene por parámetro
- GETCURRENTTIME es una acción de Tipo de Valor IO (no una función), al ser requerida, el runtime la ejecutará para obtener la fecha y hora actual

5. Acciones vs Función Pura que Retorna una Acción:

```
import Data.Time.Clock

ejemplo :: IO ()
ejemplo = do
 let x = 5
 print x

mensajeTiempo :: IO String
mensajeTiempo = do
    putStrLn "ingrese su nombre: "
    usuario <- getLine
    diahora <- getCurrentTime
    return (usuario ++ " " ++ show diahora)

mensajeTiempoBis :: String -> IO String
mensajeTiempoBis usuario = do
    diahora <- getCurrentTime
    return (usuario ++ " " ++ show diahora)

main :: IO ()
main = do
    ejemplo
    txt <- mensajeTiempo
    putStrLn (txt)
    txtBis <- mensajeTiempoBis "David"
    putStrLn (txtBis)
```

- mensajeTiempo es una acción que retorna un String, el operador <- obliga al runtime a ejecutar la acción y al valor retornado lo vincula al identificador txt
- mensajeTiempoBis es una función pura que retorna una acción, a diferencia de mensajeTiempo, recibe el nombre de usuario como parámetro

- putStrLn no puede ejecutar directamente una acción y operar sobre el valor retornado, por ello es necesario el paso previo

Inmutabilidad

Una de las bases del paradigma es evitar los cambios de estados: los datos no pueden ser modificados después de su creación. La técnica para implementar este requerimiento es la Inmutabilidad, a través de ella, ante la necesidad de realizar un cambio, se crea una nueva copia que incluya los cambios deseados, manteniendo los datos originales intactos. Esta inmutabilidad tiene ventajas importantes: permite que el código sea más predecible porque se garantiza que los datos no cambiarán inesperadamente afectando otra parte del programa evitando así efectos secundarios indeseados. Además, facilita la programación concurrente ya que múltiples procesos pueden leer los mismos datos sin riesgo de interferencia.

La inmutabilidad puede generar una ineficiencia en el uso de memoria, ya que cada modificación de una estructura de datos requiere crear una nueva versión en lugar de alterar la existente, lo que puede implicar copiar parte de la estructura y mantener en memoria versiones anteriores mientras existan referencias activas a ellas. Para mitigar esta ineficiencia, los lenguajes funcionales modernos han incorporado técnicas de optimización

Transparencia referencial

La transparencia referencial es una propiedad por la cual una expresión puede reemplazarse por su valor sin cambiar el comportamiento del programa. En programación funcional, esta propiedad está estrechamente relacionada con la inmutabilidad de los datos y la pureza de las funciones.

En Haskell el símbolo igual "=", no representa asignación tal como en lenguajes imperativos, representa *igualdad definitoria* tal como en matemática. Es decir, la expresión let x = 10 define que x representa el valor 10 (es 10), no se entiende que x contiene un 10. Entonces x no es una variable, se lo define como un *identificador inmutable*. Por lo cual, en cualquier lugar, dentro de su alcance, podría reemplazarse literalmente x por 10 gracias a la *transparencia referencial* garantizando que el significado del programa no se verá afectado.

Evaluación perezosa (lazy evaluation)

Los valores y expresiones no se calculan (no se reducen) hasta que realmente se necesitan, lo que permite mejorar la eficiencia.

Si bien la evaluación perezosa aporta beneficios importantes (como trabajar con estructuras infinitas o evitar cómputos innecesarios), puede resultar perjudicial cuando una operación acumula progresivamente cálculos pendientes, generando consumo excesivo de memoria o desbordamiento de pila al resolverse tardíamente.

1. Ejemplo

```
main :: IO()
main = do
  let x = 10 / 0
  let y = "llegué"
  putStrLn y
--el resultado es instantáneo, no intenta realizar el cociente porque nunca fue requerido
```

2. Ejemplo

```
ignorar :: Int -> String
ignorar x = "No necesito el argumento, asi que no trabajo."

main :: IO ()
main = do
   let listaInfinita = [1..]
   print (ignorar (sum (listaInfinita)))
```

```
--el resultado es instantáneo, no intenta realizar la sumatoria de los infinitos elementos de la lista (lenguajes tradicionales) porque entiende que la función Ignorar no lo requiere para dar respuesta
```

3. Ejemplo

```
main :: IO ()
main = do
   let listaInfinita = [1..]
   print (take 3 listaInfinita)
--el resultado es instantáneo, no intenta generar toda la lista de infinitos elementos (tarea imposible), se concentra en los estrictamente necesarios para dar respuesta
```

4. Ejemplo

```
-- Simula una carga de datos extremadamente pesada (ej: 2gb)
cargarArchivoGigante :: String -> String
cargarArchivoGigante path = error "¡ERROR! Sin memoria disponible"

-- Simple validación de acceso
esAdmin :: String -> Bool
esAdmin x = x == "root"

-- Función principal
accesoAlSistema :: String -> String -> String
accesoAlSistema usuario datos =
   if esAdmin usuario
   then "Acceso concedido: " ++ take 10 datos
   else "Acceso denegado. No se accedio a los datos."

main :: IO ()
main = print (accesoAlSistema "invitado" (cargarArchivoGigante "/ruta/gigante.csv"))
--el resultado es instantáneo, al detectar que el usuario no tiene acceso, no intenta recuperar el enorme volumen de datos del archivo referenciado que terminaría por agotar la memoria
```

Pattern Matching

El pattern matching es el mecanismo que, al evaluar una función, compara los argumentos recibidos con los patrones definidos, de arriba hacia abajo. La primera definición cuyo patrón coincida y su guarda (de existir) se cumpla (true) será la seleccionada y se procederá a enlazar los identificadores de los parámetros del patrón a los argumentos dados o a las partes correspondientes de su estructura.
Si habiendo recorrido todas las definiciones no se halló una cuyo patrón se ajuste a los argumentos recibidos y su guarda (de existir) se verifique, se presentará un error de ejecución.

1. Ejemplo

```
sumarOp :: Int -> Int -> Int
sumarOp x 0 = x
sumarOp 0 y = y
sumarOp x y = x + y
```

2. Ejemplo

```
sumarDos :: [Int] -> Int
sumarDos (x:y:_) = x + y -- Caso 1: La lista tiene al menos 2 elementos
sumarDos [x] = x -- Caso 2: La lista tiene exactamente 1 elemento
sumarDos [] = 0 -- Caso 3: La lista está vacía
```

```
main :: IO()
main = do
  print (sumarDos [10,7,8,23]) -- retorna 17
```

3. Ejemplo

```
cabeza :: [Int] -> Int
cabeza [] = error "lista vacía"
cabeza (x:xs) = x

main :: IO()
main = do
  print (cabeza [10,7,8,23]) -- retorna 10
```

4. Ejemplo

```
sumarOpcional :: Int -> Maybe Int -> Int -- Maybe indica que el segundo parámetro es opcional
sumarOpcional x (Just y) = x + y -- Caso 2 argumentos: Just reconoce el segundo parámetro
sumarOpcional x Nothing = x -- Caso 1 argumento: Nothing indica que el segundo parámetro no fue recibido

main :: IO()
main = do
   print (sumarOpcional 10 (Just 2)) -- retorna 12
   print (sumarOpcional 10 Nothing) -- retorna 10
```

5. Ejemplo

```
-- con enumeración
data Color = Rojo | Verde | Azul
puntaje :: Color -> Int -> Int
puntaje Rojo cantidad = cantidad * 2
puntaje Verde cantidad = cantidad * 5
puntaje Azul cantidad = cantidad * 10

main :: IO()
main = do
  print (puntaje Rojo 3)
  print (puntaje Verde 6)
  print (puntaje Azul 8)
```

6. Ejemplo

```
-- con constructores
data Color = Rojo Int | Verde Int | Azul Int

puntaje :: Color -> Int
puntaje (Rojo cantidad) = cantidad * 2
puntaje (Verde cantidad) = cantidad * 5
puntaje (Azul cantidad) = cantidad * 10

main :: IO()
main = do
  print (puntaje (Rojo 3))
  print (puntaje (Verde 6))
  print (puntaje (Azul 8))
```

El símbolo **guion bajo** se conoce como *Wilcard Pattern* (Patrón Comodín), es utilizado para indicar que el valor o parámetro que se encuentra en dicha posición debe ser ignorado. Por lo tanto, la expresión recibida en la ubicación del comodín no será evaluada. No es posible utilizarlo en el cuerpo de la función, a la derecha de la igualdad.
Es la forma idiomática en Haskell de expresar explícitamente que un argumento es **irrelevante**, siendo más preciso que usar un identificador que nunca se menciona en la expresión.

Ejemplo

```
yLogico :: Bool -> Bool -> Bool
yLogico True p = p
yLogico False _ = False
```

Recursividad

El concepto recursividad hace referencia a un mecanismo en donde una función se llama a sí misma. Esta técnica permite reemplazar las estructuras iterativas (bucles) típicas de la programación imperativa. Es fundamental en programación funcional debido a la inmutabilidad.
Algunas funciones que incorpora haskell sobre listas (ej.: foldr, filter y map) son recursivas.

1. Ejemplo con guardas

```
factorial :: Int -> Int
factorial n
      | n == 0 = 1
      | n > 0 = n * factorial (n - 1) -- alternativa: otherwise = n * factorial (n - 1)

main :: IO()
main = do
  print (factorial 3)
```

2. Ejemplo con Pattern Matching

```
factorial :: Int -> Int
factorial 0 = 1
factorial n = n * factorial (n - 1)

main :: IO()
main = do
  print (factorial 3)
```

3. Ejemplo con listas

```
-- funciones recursivas nativas que operan sobre listas
length :: [a] -> Int
length [] = 0
length (x:xs) = 1 + length xs

last :: [a] -> a
last [x] = x
last (x:xs) = last xs

take :: Int -> [a] -> [a]
take n _ | n <= 0 = []
take _ [] = []
take n (x:xs) = x : take (n - 1) xs
```

Expresiones lambda o funciones anónimas

Las funciones anónimas, como su nombre lo indica, son funciones sin nombre que se definen en el lugar donde se usan, útiles para operaciones simples y pasajes de funciones como argumentos.

La notación **λ** de Church permite la definición de funciones anónimas. Cada argumento va precedido por un símbolo **λ** y finaliza con un punto, en el último término se ubica la expresión o cuerpo de la función:

$$\lambda n.\ n^3$$

Define la función anónima donde λn identifica el argumento y n^3 la expresión.

Las siguientes expresiones tienen el mismo cuerpo, pero no son equivalentes, el orden en que se declaran los parámetros determina qué argumento recibirá: el primer argumento se asigna al primer parámetro, y el segundo al segundo.

$$\lambda n.\ \lambda m.\ n^3 - m \neq \lambda m.\ \lambda n.\ n^3 - m$$

En Haskell, la función anónima $\lambda n.\ n^3$ se define como (\n -> n^3), mientras que $\lambda n.\ \lambda m.\ n^3 - m$ se define como (\n -> \m -> n^3 - m), o en su forma abreviada (\n m -> n^3 - m).

Los parámetros se asignan de izquierda a derecha, es decir, el primer valor será asignado al primer parámetro, el segundo valor será asignado al segundo parámetro, y si sucesivamente. Ejemplificando:

- Aplicar (\n -> \m -> n^3 - m) 2 5 implicará n = 2 y m = 5 retornando 3
- Aplicar (\m -> \n -> n^3 - m) 2 5 implicará m = 2 y n = 5 retornando 123

Ejemplos:

1. (\x -> x^2) 3 -- donde (\x -> x^2) es la función anónima que se aplicará con el valor 3. Retornará: 9

2. (\x -> \y -> y - x) 2 3 -- donde (\x -> \y -> y - x) es la función anónima que se aplicará con los valores x = 2 e y = 3. Retornará: 1

3. map (\x -> x * 2) [1, 2, 3, 4, 5] -- donde (\x -> x * 2) es la función anónima que se aplicará a cada elemento de la lista. Retornará: [2, 4, 6, 8, 10]

Currificación

Es un mecanismo, popularizado por Haskell B. Curry, que permite representar una función de varios argumentos a través de varias funciones de un argumento.

En consecuencia, toda función de múltiples argumentos es en realidad una **cadena de funciones de un solo argumento**.

Como es sabido, el símbolo →, asocia hacia la derecha, esto significa que las dos expresiones siguientes son equivalentes:

- fun :: Int → Int → Int
- fun :: Int → (Int → Int)

Por lo tanto, se puede decir que *fun* es una función que recibe un entero y devuelve una función que, a su vez, recibe un entero y devuelve un entero.

Siempre que no se especifiquen los paréntesis, se supondrá que la expresión se encuentra en forma currificada.

A continuación, se muestran algunos ejemplos con la sintaxis tradicional y con la sintaxis currificada:

Matemática	Haskell
g(x)	g x
f(x,y)	f x y
g(f(x,y))	g (f x y)

f (x, g(y))	f x (g y)
g(x+y)	g (x+y)
g(x)+y	g x + y

Clarificando, durante el proceso de reducción, al tomar el primer argumento devolverá una nueva función (derivada de la original) con el primer argumento fijado. Luego procederá a fijar el segundo argumento sobre la función derivada, obteniendo una nueva función donde el primer y segundo argumento se encuentran establecidos; este proceso se repetirá por cada argumento requerido hasta aplicar el último argumento, llegando a la reducción final y obteniendo así el valor solicitado.

1. Ejemplo Proceso de Reducción

```
suma :: Int -> Int -> Int
suma a b = a + b

suma 3 4
(\a -> \b -> a + b) 3 4 -- es igualmente válido expresarla como: (\a b -> a + b) 3 4
(\b -> 3 + b) 4
3 + 4
7
```

2. Ejemplo Proceso de Reducción

```
producto :: Int -> Int -> Int -> Int
producto a b c = a * b * c

producto 3 4 5
(\a -> \b -> \c -> a * b * c) 3 4 5 -- es igualmente válido expresarla como: (\a b c -> a * b * c) 3 4 5
(\b -> \c -> 3 * b * c) 4 5 -- es igualmente válido expresarla como: (\b c -> 3 * b * c) 4 5
(\c -> 3 * 4 * c) 5
3 * 4 * 5
60
```

Aplicación Parcial

Es la consecuencia directa de la currificación: como cada función espera un argumento a la vez, es posible aplicarla con menos argumentos de los requeridos.
Hay aplicación parcial cuando se proporciona a una función al menos un argumento, pero menos de los que espera; el resultado es una nueva función que aguarda los restantes.
Si no se le proporciona ningún argumento, se trata de la misma función con otro nombre, es decir, un sinónimo (por ejemplo, *longitud = length*, que resulta de aplicar reducción eta a *longitud xs = length xs*).
Una función que espera un solo argumento solo admite una aplicación completa.

Reducción eta *es una regla del cálculo lambda que plantea lo siguiente: si en la definición de una función el argumento aparece* ***solo al final*** *de ambos términos (no se menciona en ningún otro lugar de la expresión), el argumento puede ser suprimido.*

1. Ejemplo

```
producto :: Int -> Int -> Int
producto x y = x * y

-- aplicación parcial sobre la función producto (aplica), y reducción eta sobre la función doble (define)
doble :: Int -> Int
doble = producto 2
```

```
main :: IO()
main = print (doble 3)
-- resultado: 6
```

2. Ejemplo

```
producto :: Int -> Int -> Int
producto x y = x * y

-- aplicación parcial sobre la función producto (aplica),
  hay reducción eta sobre la función multiplicarPorDiez (define)
main :: IO ()
main = do
   let multiplicarPorDiez = producto 10
   print (multiplicarPorDiez 4)
```

3. Ejemplo

```
producto :: Int -> Int -> Int -> Int
producto a b c = a * b * c

-- aplicación parcial de la función producto (aplica), sin reducción eta
multiplicarLista :: [Int] -> [Int]
multiplicarLista lista = map (producto 3 4) lista

-- aplicación parcial de la función producto (aplica), aplicación parcial de la función map (aplica),
  y reducción eta sobre multiplicarListaEta (define)
multiplicarListaEta :: [Int] -> [Int]
multiplicarListaEta = map (producto 3 4)

main :: IO()
main = print (multiplicarLista [1, 2, 3, 4, 5])
-- equivale a aplicar (\c -> 12 * c) a cada elemento
-- resultado: [12, 24, 36, 48, 60]
```

Funciones de orden superior

Son funciones que reciben como parámetros otras funciones o devuelven funciones como resultado.
En Haskell, como toda función es pura, las funciones de orden superior también lo son.

1. Función Orden Superior que recibe una función, implementando funciones nombradas:

```
sumar :: Int -> Int -> Int
sumar a b = a + b

restar :: Int -> Int -> Int
restar a b = a - b

operar :: (Int -> Int -> Int) -> Int -> Int -> String
operar unaFun n1 n2 = show (unaFun n1 n2)

main :: IO ()
main = do
    let totalSumado = operar sumar 2 3
    let totalRestado = operar restar 9 2
```

```
    putStrLn ("Suma: " ++ totalSumado)
    putStrLn ("Diferencia: " ++ totalRestado)
```

2. Función Orden Superior que recibe una función, implementando funciones anónimas:

```
operar :: (Int -> Int -> Int) -> Int -> Int -> String
operar unaFun n1 n2 = show (unaFun n1 n2)

main :: IO ()
main = do
    let totalSumado = operar (\x y -> x + y) 2 3
    let totalRestado = operar (\x y -> x - y) 9 2
    putStrLn ("Suma: " ++ totalSumado)
    putStrLn ("Diferencia: " ++ totalRestado)
-- Se visualiza que las funciones pueden ser definidas al momento de requerirlas.
```

3. Función Orden Superior que recibe y retorna una función:

```
triplicar :: Int -> Int
triplicar x = x * 3

aplicarDosVeces :: (Int -> Int) -> (Int -> Int)
aplicarDosVeces f = \x -> f (f x) -- recibe la función f y retorna una función lambda con ella

main :: IO ()
main = do
    print (aplicarDosVeces triplicar 4)

-- Proceso de evaluación
aplicarDosVeces triplicar 4
(\x -> triplicar (triplicar x)) 4
triplicar (triplicar 4)
triplicar 12
36
```

Composición de funciones

Consiste en combinar funciones simples para crear funciones más complejas, ej.: H(x) = F(G(x)) donde H(x) está definida como F(G(x)), por lo cual, ejemplificando: el valor obtenido al realizar H(2) será el mismo que al realizar F(G(2)). Ante modificaciones en F y/o G, H reflejará esos cambios automáticamente ya que se trabaja con referencias a dichas funciones.

La composición es posible expresarla a través del operador punto ".":

f (g x) = (f . g) x

En ambos casos, primero se aplicará la función **g** y luego la función **f**.

Al utilizar el operador punto, solo se gana en claridad semántica.

Ejemplificando, **procesar x = f (g (h (i x)))**, es posible plantearla como **procesar x = (f . g . h . i) x** para luego, a través de la *reducción eta* poder expresarla como **procesar = f . g . h . i** obteniendo así una expresión mucho más clara.

1. Ejemplo

Si se define la función:

```
doble :: Int -> Int
doble x = 2 * x
```

Es posible definir la función cuádruple como:

```
cuadruple w = (doble . doble) w -- la función cuádruple está compuesta por dos funciones doble
```

Tal como fue planteado en la definición de composición, el operador (.) se define como (f . g) x = f (g x), entonces:

```
cuadruple w = doble (doble w)
```

Ejemplificando los pasos de evaluación:
cuadruple 5
doble (doble 5)
doble (2 * 5)
doble 10
2 * 10
20

2. Ejemplo
Se define una función compuesta que permite conocer si una palabra dada tiene una cantidad par de caracteres:

```
palabraPar :: String -> Bool
palabraPar texto = (even . length) texto
```

Definir palabraPar como palabraPar texto = even . (length texto) sería incorrecto: la composición es una operación entre funciones, de hacerlo, se pretendería realizar una composición entre una función y el valor retornado por otra (length).

Aplicando reducción eta, se expresaría:

```
palabraPar = even . length
```

3. Ejemplo
Se define una función compuesta denominada formula a partir de funciones definidas:

```
subirUno :: Int -> Int
subirUno x = x + 1

duplicar :: Int -> Int
duplicar x = x * 2

formula :: Int -> Int
formula = duplicar . subirUno

main :: IO ()
main = do
    let resultado = formula 5
    print ("Valor Final: " ++ show resultado)
```

4. Ejemplo
Se definen dos funciones compuestas que permiten obtener los mismos resultados con diferentes usos de la cláusula where:

```
--Se define el mensaje a presentar a partir del valor que recibe mostrarMensaje
--(even retorna un bool en función del nro retornado por length)
tieneCantidadPar :: [String] -> String
tieneCantidadPar = mostrarMensaje . even . length
  where
    mostrarMensaje True  = "La lista tiene un numero par de elementos."
```

```
        mostrarMensaje False = "La lista tiene un numero impar de elementos."

tieneCantidadParConIf :: [String] -> String
tieneCantidadParConIf = mostrarMensaje . even . length
    where
        mostrarMensaje b =   if b then
                                "La lista tiene un numero par de elementos."
                             else
                                "La lista tiene un numero impar de elementos."
        main :: IO ()
        main = putStrLn (tieneCantidadPar ["Habia", "una", "vez"] ++ "\n"
        ++ tieneCantidadParConIf ["una", "casa"])

        --Salida:
        --La lista tiene un numero impar de elementos.
        --La lista tiene un numero par de elementos.
```

Analizando la función foldr:
La función **foldr** es, por definición, recursiva y de orden superior. La evaluación perezosa, propia de Haskell, se puede observar en los ejemplos 3 y 4, donde se opera sobre listas infinitas. En el ejemplo 3 el cálculo finaliza porque **(&&)** con **False** como primer argumento no necesita evaluar el segundo. En el ejemplo 4 no finaliza, porque **(+)** necesita el valor de su segundo argumento, el cual es otro **foldr** aplicado sobre el resto de la lista, que continúa siendo infinita. En consecuencia, las sumas van quedando pendientes indefinidamente.

Definición de Haskell:

```
foldr f z [] = z
foldr f z (x:xs) = f x (foldr f z xs)
```

1. Ejemplo de Aplicación:

```
foldr (\x acu -> x ++ acu) "" ["hola", "que", "tal"]
```

Pasos de Evaluación:

1. foldr (\x acu -> x ++ acu) "" ["hola", "que", "tal"]
2. "hola" ++ (foldr (\x acu -> x ++ acu) "" ["que", "tal"])
3. "hola" ++ ("que" ++ (foldr (\x acu -> x ++ acu) "" ["tal"]))
4. "hola" ++ ("que" ++ ("tal" ++ (foldr (\x acu -> x ++ acu) "" [])))
5. "hola" ++ ("que" ++ ("tal" ++ ""))
6. "hola" ++ ("que" ++ "tal")
7. "hola" ++ "quetal"
8. "holaquetal"

Salida:
"holaquetal"

2. Ejemplo de Aplicación:

```
foldr (-) 0 [50,-40,300]
```

Pasos de Evaluación:

1. foldr (-) 0 [50,-40,300]
2. 50 - (foldr (-) 0 [-40, 300])
3. 50 - ((-40) - (foldr (-) 0 [300]))
4. 50 - ((-40) - (300 - (foldr (-) 0 [])))
5. 50 - ((-40) - (300 - (0)))

6. 50 - ((-40) - (300))
7. 50 - (-340)
8. 390

Salida:
390

3. Ejemplo de Aplicación:
foldr (&&) True (repeat False) --repeat genera una lista infinita con el valor indicado en todas sus posiciones

Pasos de Evaluación:
1. foldr (&&) True (repeat False)
2. foldr (&&) True (False : repeat False) -- repeat x = x : repeat x
3. (&&) False (foldr (&&) True (repeat False)) -- (&&) False _ = False, el segundo argumento no se evalúa
4. False

Se utilizó la notación prefija del operador && pero se obtendría la misma conclusión al utilizarlo como operador infijo:

1. foldr (&&) True (repeat False)
2. foldr (&&) True (False : repeat False) -- repeat x = x : repeat x
3. False && (foldr (&&) True (repeat False)) -- False && _ = False, el segundo argumento no se evalúa
4. False

Salida:
False -- Aunque la lista es infinita, el cálculo termina: solo se necesitó inspeccionar su primer elemento, ***el foldr restante nunca se evalúa***

4. Ejemplo de Aplicación:
foldr (+) 0 [1..]

Pasos de Evaluación:
1. foldr (+) 0 [1..]
2. (+) 1 (foldr (+) 0 [2..])
3. (+) 1 ((+) 2 (foldr (+) 0 [3..]))
4. ...

Salida:
Nunca finaliza

Compiladores online útiles para testear código:

- https://play.haskell.org/ (no acepta ingresos del usuario)
- https://www.onlinegdb.com/#

Contenido

Paradigma Funcional 1
Tipos de Datos 1
Tipos Primitivos 1
Tipos Especiales 1
Listas 2
Tuplas 3
Tipos Personalizados 4
Funciones Predefinidas Básicas 7
Funciones de Conversión Básicas 8
Algunas Reglas Sintácticas 9
Vocabulario 9
Forma Normal (NF) o Irreductibilidad 9
Aplicación 10
Evaluación 10
Definición de Funciones 10
Instrucción Let 12
Cláusula Where 13
Conceptos 14
Funciones puras 14
Inmutabilidad 17
Transparencia referencial 17
Evaluación perezosa (lazy evaluation) 17
Pattern Matching 18
Recursividad 20
Expresiones lambda o funciones anónimas 21
Currificación 21
Aplicación Parcial 22
Funciones de orden superior 23
Composición de funciones 24